

```
[ColumnName("Score")]
public float[] Score;
}
```

The *MNISTData* data class is our input data class. It contains the definitions of our features and the labels. We are using the *Column* attribute and a range of 0-63 to treat our 64 columns as an array of floats and are calling it *PixelValues*. We are also defining a size of 64 for these columns since the ML.NET trainer requires vectors of a known size. Next up is our label in the last column that we call *Number*.

The *NumberPrediction* class is used for the output of our ML model. The *PredictedLabel* column is of *uint* type and is called *NumberLabel*, and the *Score* column is a float array called *Score*.

Creating a training workflow

With these basics in place, we can move ahead with building our ML pipeline. Remove or comment the 'Hello World!' line from the *Main()* method and add the following lines:

```
var env = new LocalEnvironment();
var model = Train(env);
```

We are creating an ML environment using *LocalEnvironment* and passing that to the *Train* method. Add the *Train()* method after the *Main()* method with the following code:

```
public static dynamic Train(LocalEnvironment env)
{
    try
    {
        var classification = new MulticlassClassificationContext(env);
        var reader = TextLoader.CreateReader(env, ctx => (
            PixelValues: ctx.LoadFloat(0, 63),
            Number: ctx.LoadText(64)
        ),
        separator: ',',
        hasHeader: false);

        var learningPipeline = reader.
            MakeNewEstimator()
                .Append(r => (
                    label: r.Number.ToKey(),
                    Features: r.PixelValues))
                .Append(r => (
                    r.label,
                    Predictions: classification.Trainers.
                        MultiClassLogisticRegression(r.label, r.Features)))
                .Append(r => (
                    PredictedLabel: r.Predictions.
```

```
predictedLabel.ToValue(),
                    Score: r.Predictions.score));

        var data = reader.Read(new MultiFileSource(_
datapath));
        var model = learningPipeline.Fit(data).
            AsDynamic();
        return model;
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
        return null;
    }
}
```

LocalEnvironment is a class that creates an ML.NET environment for local execution. In this method, we are creating a classification context of type *MultiClassClassificationContext()* since our little project here deals with classifying the images as one out of ten numbers. We then use the *TextLoader* class to point to our data files and the structure of the data as explained in the data processing section of this article. The *TextLoader* class performs lazy reading, so the data will not be read until we perform a *Read* operation just before the training.

The learning pipeline is built using the *MakeNewEstimator()* class. An estimator is a class that represents the attributes related to an ML algorithm. We start by appending our label and features as the *Number* and *PixelValues* as defined in our input data classes, respectively. We now add the *MultiClassLogisticRegression* trainer to the pipeline and finally, define the structure of the predictions that we expect.

After this is done, we read the data from our training data file and perform the training task by calling the *Fit* method. This step will produce a model that we can save and reuse for making predictions with our testing and/or real-world inputs.

You can run this program now to see how the training task is performed by running *dotnet run* in the project directory. If no errors pop up during the execution of this program, our training was successful; our model will be returned, and can be saved for reuse later.

You can add the following code to the *Main()* method to save the file to disk:

```
using (var stream = File.Create(_modelpath))
{
    model.SaveTo(env, stream);
}
```

After these changes, you should be able to run the program, and see the model file saved to the file system for reuse, if needed.